CRIPTA

Proyecto de Estructuras de Datos – II Ciclo 2026 · Segunda parte

Rutas ponderadas, conocimiento parcial del mapa, verificación de resolubilidad y línea de tiempo, desarrolladas con especificaciones ejecutables

Aspecto	Definición
Modalidad	Los mismos grupos de la primera parte (hasta 4 personas)
Base de código	Se continúa sobre el repositorio de la primera parte
Lenguaje de programación	Python 3.11 o superior
Metodología	Desarrollo dirigido por especificaciones (SDD) con GitHub spec-kit (§2)
Porcentaje	15%
Uso de LLM	Permitido, con bitácora obligatoria de prompts; incluye los comandos de spec-kit (§2.6)
Fecha de entrega	Miércoles 18 de noviembre

Regla editorial del proyecto: las reglas del juego están especificadas; las decisiones de estructuras de datos no. En esta parte, además, las especificaciones las escriben ustedes antes que el código.

Todo lo establecido en la primera parte sigue vigente salvo que este documento lo modifique explícitamente. Las referencias de la forma «P1 §x» remiten al enunciado de la primera parte.

1. Propósito de la segunda parte

La primera parte limitó deliberadamente lo que los actores saben del mapa: el rastreador solo mira a los cuartos vecinos y no busca rutas. Esta parte retira ese límite e introduce cuatro capacidades nuevas:

1. **Terreno y cazadores.** Las salidas tienen costo de tránsito y aparece un enemigo que calcula la ruta más corta hacia el jugador.
2. **Niebla de guerra.** El jugador solo conoce la parte del mapa que ha explorado, y puede viajar automáticamente dentro de ella.
3. **Verificación de resolubilidad.** Antes de jugar, el programa determina si la cripta puede ganarse y, si no, explica por qué.
4. **Línea de tiempo.** Se puede consultar el estado de la partida en cualquier instante pasado, sin copiar el estado completo en cada acción.

Como en la primera parte, no se indica qué estructura usar. Se describen operaciones, propiedades y casos borde; la elección y su defensa con mediciones son parte de la evaluación.

La novedad es metodológica: cada capacidad debe especificarse, planificarse y descomponerse en tareas con spec-kit **antes** de implementarse. Una especificación que no describe lo que el programa hace, o un programa que hace cosas que ninguna especificación describe, son defectos del mismo tipo.

> **Importante.** Las decisiones de la primera parte pueden dejar de ser adecuadas con las reglas nuevas. Detectarlo, medirlo y documentar el cambio en el Documento de Decisiones es parte esperada del trabajo, no una señal de error.

2. Desarrollo dirigido por especificaciones con spec-kit

Cada capacidad nueva se desarrolla con el flujo de GitHub spec-kit: primero se escribe qué debe hacer el programa, luego cómo se hará, luego las tareas, y solo entonces el código. Los artefactos de spec-kit son Markdown versionado dentro del repositorio y se evalúan como cualquier otro entregable.

2.1 Instalación

spec-kit se instala con `uv` y se inicializa sobre el repositorio existente de la primera parte:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@vX.Y.Z
specify init --here --ai <agente>
```

- Fijen una versión concreta (`@vX.Y.Z`) y regístrenla en el README. Todo el grupo usa la misma.
- El agente de programación es decisión del grupo (Claude Code, GitHub Copilot, Gemini CLI u otro compatible). Indíquenlo en el README.
- Los archivos que genera `specify init` (`.specify/`, comandos del agente) se versionan en el repositorio.

2.2 Flujo obligatorio por capacidad

1. `/speckit.constitution` — una sola vez al inicio: principios que gobiernan todo el proyecto (§2.3).
2. `/speckit.specify` — qué debe hacer la capacidad y por qué, desde el punto de vista del juego. Produce `specs/<NNN-nombre>/spec.md`.
3. `/speckit.clarify` — resuelve ambigüedades de la especificación. Las preguntas y respuestas quedan registradas en la especificación.

4. `/speckit.plan` — cómo se implementará: estructuras de datos, módulos afectados, mediciones previstas. Produce `plan.md` y artefactos asociados (`research.md`, `data-model.md`).
5. `/speckit.tasks` — descompone el plan en tareas verificables. Produce `tasks.md`.
6. `/speckit.analyze` — revisión de consistencia entre especificación, plan y tareas antes de implementar.
7. `/speckit.implement` — ejecución de las tareas. Pueden implementar manualmente, con el agente o combinando ambos.

Como mínimo debe haber una especificación por cada capacidad de §3: terreno y cazador, niebla de guerra (incluido «ir a»), resolubilidad y línea de tiempo. Pueden dividir una capacidad en varias especificaciones si lo justifican.

2.3 Constitución

`.specify/memory/constitution.md` debe recoger, en palabras del grupo, al menos estos principios heredados del proyecto:

- Las estructuras de datos de los problemas evaluados se implementan a mano; está prohibido `dict` como índice o caché y las implementaciones equivalentes de bibliotecas (P1 §7 y §6.2 de este documento).
- Determinismo: misma cripta, misma semilla y mismas acciones producen la misma partida.
- Separación MVC; `--replay` y `--bench` no instancian la vista.
- Ningún dato de una cripta concreta escrito en el código.
- Respeto de `presupuesto_solicitudes`.
- Toda afirmación de rendimiento se respalda con `--bench`.
- Una especificación precede a su implementación.

Los agentes leen la constitución en cada fase. Si un plan o una tarea la contradice, el error es del grupo, no del agente.

2.4 ¿Qué va en la especificación y qué va en el plan?

La separación de spec-kit coincide con la regla editorial de este curso:

Artefacto	Contiene	No contiene
`spec.md`	Reglas del juego, historias de usuario, escenarios de aceptación, requisitos funcionales y de rendimiento numerados	Nombres de estructuras de datos, clases o módulos
`plan.md`	Estructuras elegidas, alternativas consideradas, complejidad esperada, módulos afectados, mediciones planificadas	Reglas nuevas que no estén en `spec.md`
`tasks.md`	Tareas pequeñas y verificables, con dependencias y referencia al requisito que atienden	Tareas sin requisito asociado

El `plan.md` de cada capacidad es el borrador de la sección correspondiente del Documento de Decisiones (§7.1). No es necesario duplicar el texto: el Documento puede enlazar al plan, pero las mediciones finales van en el Documento.

2.5 Trazabilidad

- Cada requisito funcional tiene un identificador estable (por ejemplo `FR-012`).

- Cada caso del Apéndice B de este documento aparece como escenario de aceptación en alguna especificación.
- Cada escenario de aceptación tiene al menos una prueba automatizada o un `partida.log` de reproducción que lo ejercita. La prueba cita el identificador del requisito.
- Cada tarea de `tasks.md` referencia los requisitos que atiende.

En la defensa se escogerá un requisito al azar y el grupo deberá mostrar su escenario, su tarea, su código y su prueba.

2.6 Historial, deriva y bitácora de LLM

- El historial de Git debe mostrar la especificación de cada capacidad confirmada antes que los cambios de código que la implementan.
- Si durante la implementación descubren que la especificación estaba mal, se corrige primero la especificación y luego el código, en confirmaciones separadas. Una especificación desactualizada respecto del programa entregado se penaliza.
- Cada invocación de un comando de spec-kit con texto propio (`/speckit.specify …`, `/speckit.clarify`, etc.) es un prompt a un LLM y se registra en `PROMPTS_LLM.md` como indica P1 §10.3.

3. Reglas nuevas de Cripta

Esta sección amplía las reglas de P1 §2. Como entonces, todos los programas deben producir el mismo resultado observable ante la misma cripta, semilla y secuencia de acciones.

3.1 Terreno y costo de las salidas

Cada salida puede declarar un campo `costo`, entero mayor o igual que 1. Si no lo declara, su costo es 100. El costo es el mismo en ambos sentidos.

Moverse a través de una salida es una acción cuyo costo base es el `costo` de esa salida, en lugar del valor fijo 100 de P1 §2.3. El intervalo se sigue calculando con la fórmula de P1:

```
intervalo = max(1, costo_salida * 100 // velocidad)
```

La regla aplica a todos los actores. El costo de atacar, esperar y las demás acciones no cambia. Errantes y rastreadores conservan su comportamiento; solo cambia lo que tarda su movimiento.

3.2 El cazador

Se agrega un cuarto comportamiento de enemigo, `cazador`. Su ficha de catálogo incluye el campo `rango`, un entero en unidades de costo.

En su turno, el cazador aplica estas reglas en orden:

1. Si comparte sala con el jugador, ataca (P1 §2.8).
2. Si no, considera el costo mínimo `d` de llegar desde su sala hasta la sala actual del jugador, sumando el `costo` de cada salida recorrida y usando únicamente salidas abiertas en ese instante.
3. Si no existe ruta, o si `d > rango`, espera (costo 100).
4. Si existe, se mueve un paso por una ruta de costo `d`. El costo de esa acción es el `costo` de la salida que atraviesa.

Desempate. Sea `u` la sala del cazador. Entre las vecinas `v` tales que `costo(u, v) + dist(v, jugador) = d`, el cazador se mueve a la de menor id. Este criterio define el resultado sin importar el algoritmo que usen para calcularlo.

Conocimiento. El cazador conoce el esqueleto completo y el estado actual de todas las puertas. No le afecta la niebla de guerra del jugador. Como cualquier enemigo, no abre puertas cerradas. Su cálculo depende únicamente del esqueleto, que se descarga completo al iniciar la cripta e incluye el costo de cada salida, y del estado actual de las puertas; nunca de si el contenido de una sala ya fue descargado.

Equivalencia observable. La decisión del cazador en cada turno debe ser idéntica a la que se obtendría calculando la ruta desde cero en ese instante. Las puertas que se abren o cierran, el movimiento del jugador y la muerte de otros actores pueden cambiar la respuesta entre un turno y el siguiente. Pueden reutilizar cálculos anteriores, pero nunca a costa de una decisión distinta.

3.3 Niebla de guerra

El jugador ya no ve el mapa completo. Para el jugador, cada sala está en uno de tres estados:

Estado	Cuándo	Qué conoce el jugador
Explorada	El jugador entró a ella al menos una vez. La sala inicial está explorada desde el instante 0.	Nombre, salidas, costo de cada salida y el estado de sus puertas según la última observación
Avistada	No está explorada, pero es destino de una salida de una sala explorada	Solo que existe una sala detrás de esa salida
Desconocida	Ninguna de las anteriores	Nada

Memoria de puertas. El jugador observa una puerta cuando está en cualquiera de las dos salas que conecta. El mapa del jugador recuerda el estado de cada puerta tal como estaba en su última observación, aunque haya cambiado después; por ejemplo, una puerta con cierre automático puede figurar abierta en el mapa y estar cerrada en realidad.

Antorcha. La antorcha de P1 §2.7 adquiere efecto: mientras esté encendida, al entrar a una sala también quedan exploradas todas sus vecinas, a través de cualquier salida, abierta o cerrada. Explorar una vecina de esta forma revela su nombre y salidas, no su contenido, y no activa sus enemigos ni sus trampas.

Separación entre mapa y conocimiento. El programa sigue descargando el esqueleto completo, y la estrategia de precarga (P1 §4.2) puede usarlo libremente: es infraestructura. En cambio, todo lo que la vista muestra y lo que usa el comando «ir a» (§3.4) debe derivarse exclusivamente del conocimiento del jugador.

El conocimiento del jugador es estado del juego: se guarda en el archivo de partida, se revierte con el pergamino de retroceso y forma parte de la línea de tiempo (§3.6).

La interfaz debe ofrecer un comando para ver el mapa conocido: salas exploradas, sus salidas y las salas avistadas.

3.4 Viaje automático: «ir a»

El comando `ir a <sala>` lleva al jugador a una sala explorada mediante una secuencia de movimientos ordinarios.

- La ruta se calcula con el conocimiento del jugador: solo salas exploradas y solo salidas que el mapa del jugador recuerda abiertas. Se minimiza la suma de costos y se desempata igual que el cazador (§3.2): primer paso hacia la vecina de menor id.
- Cada paso es una acción del jugador completa: consume tiempo según §3.1 y entre paso y paso ocurren normalmente los eventos del reloj.

- La ruta se recalcula antes de cada paso con el conocimiento vigente. Si al llegar a una sala el jugador observa que una puerta de su ruta está cerrada, el siguiente cálculo la evita.
- El viaje se detiene cuando: el jugador llega al destino; entra a una sala que contiene enemigos, activos o dormidos; recibe daño; o deja de existir una ruta conocida.
- No abre puertas aunque el jugador tenga la llave.
- En `partida.log`, cada paso se registra como un movimiento individual. La orden `ir a` se registra además como comentario para fines de depuración.
- El pergamino de retroceso revierte únicamente el último paso, porque cada paso es una acción del jugador.

3.5 Verificación de resolubilidad

Antes de iniciar una partida, el programa determina si la cripta puede ganarse. Para eso usa el esqueleto y la ubicación de las llaves (§4.2).

Supuestos de la verificación. El jugador puede atravesar cualquier salida abierta y cualquier salida cerrada cuya llave ya obtuvo; las llaves no se consumen; todo enemigo puede ser derrotado; una llave que suelta un enemigo se considera obtenible en la sala donde ese enemigo inicia; las trampas y los cierres automáticos no bloquean, porque una puerta puede volver a abrirse con la misma llave.

Resoluble. Una cripta es resoluble si, partiendo de la sala inicial y bajo esos supuestos, el jugador puede obtener `llave_salida` y llegar a `sala_salida` a través de una puerta que la requiere.

Resultado esperado:

- Si es resoluble: un orden de obtención de llaves en el que cada llave aparece después de todas las llaves necesarias para alcanzarla. Si hay varios órdenes válidos, cualquiera sirve, pero debe ser determinista.
- Si no lo es: las llaves que no pueden obtenerse; las puertas de la frontera bloqueada, es decir, las puertas cerradas adyacentes a la región alcanzable, con la llave que requiere cada una; y dónde se encuentra cada una de esas llaves. Si una llave está ubicada detrás de puertas que solo ella abre, directamente o mediante una cadena de llaves, el reporte debe señalar ese ciclo.

Si la cripta no es resoluble, la partida no inicia salvo que se use la bandera `--forzar`. La bandera `--verificar <cripta>` ejecuta únicamente la verificación, sin instanciar la vista.

3.6 Línea de tiempo

El pergamino de retroceso (P1 §2.11) sigue limitado a 5 acciones. La línea de tiempo es independiente: permite **consultar**, sin modificar la partida, el estado en cualquier instante desde el inicio.

Estado en el instante t. Es el estado después de ejecutar todos los eventos con tiempo menor o igual que `t`. Si a `t` le corresponde una decisión del jugador, es el estado justo antes de tomarla.

Consulta en partida. El comando `historia <t>` o `historia #k` (estado justo antes de la acción número `k` del jugador) muestra como mínimo: reloj, sala del jugador, vida y estadísticas, inventario, efectos activos, enemigos de esa sala con su vida y el mapa conocido en ese momento. La consulta no consume tiempo virtual, no se registra como acción y no altera ningún estado del juego ni del historial de retroceso.

Consulta sin interacción. `--replay partida.log --hasta <t>` reproduce la partida hasta `t` e imprime el mismo resumen sin instanciar la vista.

Interacción con el retroceso. Las acciones revertidas con un pergamino dejan de existir en la línea de tiempo: la línea describe la historia de la partida tal como quedó. Consultar un instante posterior al reloj actual se rechaza.

Restricción de memoria. Se permite guardar copias completas del estado, llamadas puntos de control, pero no más de una cada `K` acciones del jugador. `K` se configura con `--checkpoint-cada K`; el valor por defecto lo escoge el grupo y lo justifica con mediciones. Una consulta no puede requerir aplicar o deshacer cambios de más de `K` acciones del jugador. Fuera de los puntos de control, solo se almacena la información necesaria para reconstruir los cambios.

Pueden reconstruir hacia adelante desde un punto de control, re-simulando, o hacia atrás desde el estado actual, invirtiendo cambios. Si re-simulan, el punto de control debe incluir el estado del generador `random.Random`.

Persistencia. Guardar y cargar la partida conserva la línea de tiempo completa. El formato binario cambia; un archivo de la primera parte debe detectarse como incompatible (P1 §4.10) o migrarse de forma explícita.

4. Cambios al servicio web

El servicio mantiene el contrato de la primera parte y agrega los campos y la consulta siguientes. Swagger sigue siendo la referencia del contrato HTTP exacto.

4.1 Campos nuevos

Costo en las salidas del esqueleto:

```
"salidas": {
  "N": { "sala": 3 },
  "E": { "sala": 7, "costo": 250, "cerrada": true,
         "llave": "itm_llave_bronce", "cierre_automatico": 500 }
}
```

Ficha de un cazador en el catálogo:

```
{ "id": "ent_sabueso_oseo", "clase": "enemigo", "nombre": "Sabueso óseo",
  "vida_max": 18, "ataque": 6, "defensa": 2, "velocidad": 120,
  "comportamiento": "cazador", "rango": 600, "suelta": [] }
```

4.2 Ubicación de llaves

```
GET /criptas/cripta-01/llaves
{
  "llaves": [
    { "id": "itm_llave_bronce", "sala": 4 },
    { "id": "itm_llave_negra", "sala": 12, "enemigo": "e-1203" }
  ]
}
```

Cuando existe el campo `enemigo`, la llave está en el campo `suelta` de esa instancia, y `sala` es la sala donde el enemigo inicia. La respuesta es válida mientras coincida la versión de la cripta y puede persistirse con el resto de los datos de la cripta.

4.3 Consideraciones

- La consulta de llaves cuenta dentro de `presupuesto_solicitudes`, como toda solicitud.
- El servicio publicará criptas no resolubles. Conocer de antemano cuáles son no ayuda: en la revisión se usará una cripta nueva.
- Las criptas de la segunda parte tienen versiones nuevas. Lo persistido por la primera parte debe invalidarse mediante la validación de versión ya implementada, no borrando archivos a mano.

5. Problemas de estructuras de datos a resolver

Cada subsección describe un problema y las propiedades exigidas. No prescribe la estructura. Cada problema debe tener su especificación (§2) y su sección en el Documento de Decisiones.

5.1 La ruta más barata del cazador

- Los costos de las salidas no son uniformes; contar salas no basta para encontrar la ruta más barata.
- La decisión de un cazador cuyo `rango` es pequeño no debe costar proporcionalmente al tamaño de la cripta, sino a la región que queda dentro de su rango.
- En las pruebas habrá decenas de cazadores activos a la vez, y todos persiguen al mismo objetivo. Deben medir si conviene un cálculo por cazador o un cálculo compartido, y bajo qué condiciones cambia la respuesta.
- Toda estructura auxiliar que la búsqueda necesite (por ejemplo, para escoger la siguiente sala a examinar o registrar distancias) es de implementación propia. Pueden reutilizar estructuras de la primera parte si son adecuadas.
- El desempate de §3.2 debe cumplirse siempre, incluso cuando hay muchas rutas de igual costo.

5.2 Rutas que dejan de ser válidas

- Entre dos turnos de un cazador pueden ocurrir: aperturas y cierres de puertas, movimientos del jugador, cambios de sala de otros actores y cambios de velocidad.
- La decisión debe ser siempre la que daría un cálculo desde cero (§3.2). Reutilizar un cálculo anterior es válido solo si pueden demostrar que no cambia la respuesta.
- Identifiquen qué eventos invalidan qué información y cuánto trabajo se ahorra realmente.
- El `--bench` debe incluir al menos un escenario donde las puertas cambian con frecuencia y otro donde casi nunca cambian. Deben medir la alternativa de recalcular siempre.

5.3 El mapa conocido

- Consultar si una sala está explorada, avistada o desconocida y el estado recordado de una puerta son operaciones frecuentes: ocurren en cada vista, en cada paso de «ir a» y en cada actualización por antorcha.
- Explorar nuevas salas debe costar proporcionalmente a las salas reveladas, no al tamaño del mapa conocido.
- El mapa conocido se revierte con el pergamino y se reconstruye en la línea de tiempo. Su diseño debe hacer barato registrar e invertir cambios.
- La ruta de «ir a» se calcula sobre el subgrafo conocido. Justifiquen si comparten implementación con la búsqueda del cazador o si conviene separarlas.

5.4 Resolubilidad

- Obtener una llave puede volver alcanzable una región entera. Repetir el recorrido completo de la cripta cada vez que se obtiene una llave no cumple el requisito: el costo total debe crecer aproximadamente de forma lineal con salas, salidas y llaves.
- El reporte de una cripta no resoluble requiere conocer la frontera bloqueada y las dependencias entre llaves. Esa información debe salir del mismo recorrido, no de recorridos adicionales por cada llave.
- La detección de ciclos de dependencia debe ser determinista y reportar el ciclo concreto.

- El `--bench` debe medir la alternativa ingenua de repetir el recorrido por llave, con criptas sintéticas de muchas llaves encadenadas.

5.5 Línea de tiempo y puntos de control

- La consulta de un instante arbitrario debe aplicar o deshacer cambios de, como máximo, `K` acciones del jugador.
- La memoria total debe crecer con la cantidad de cambios ocurridos, más un punto de control cada `K` acciones; nunca con «acciones por tamaño del estado».
- Ubicar el punto de control más cercano a un instante pedido es una búsqueda frecuente; su costo importa en partidas largas.
- Al usar un pergamino, la línea debe recortarse sin recorrer toda la historia.
- Deben mostrar con mediciones cómo varían memoria y tiempo de consulta para al menos tres valores de `K`, y justificar el valor por defecto.
- Reconstruir hacia adelante o hacia atrás (§3.6) es una decisión de diseño que debe documentarse con evidencia.

5.6 Revisión de decisiones de la primera parte

Los cazadores, los costos variables de movimiento y la línea de tiempo cambian la carga sobre estructuras ya existentes, en especial la agenda de eventos (P1 §4.1) y el historial de retroceso (P1 §4.6). Repitan las mediciones de la primera parte con la carga nueva. Si alguna decisión deja de ser adecuada, cámbienla y documenten el antes y el después; si sigue siendo adecuada, muéstrenlo con datos.

6. Rendimiento, restricciones y medición

6.1 Presupuestos de procesamiento

Se mantienen el presupuesto de red de P1 §5.1 y el límite de 20 ms por ciclo de eventos de P1 §5.2, ahora con cazadores activos. Los nuevos límites se miden en el equipo local, sin red ni renderizado:

Operación	Límite	Condiciones
Ciclo de eventos entre dos decisiones del jugador	20 ms	Cripta de referencia con decenas de cazadores activos
Verificación de resolubilidad	200 ms	La cripta publicada más grande
Consulta `historia`	50 ms	Partida de al menos 5 000 acciones del jugador, `K` por defecto
Cálculo de un paso de «ir a»	5 ms	Mapa conocido completo de la cripta de referencia

Los valores pueden ajustarse cuando se publique la cripta de referencia; el ajuste se comunicará como enmienda (§2.7).

6.2 Restricciones de implementación

Sigue vigente P1 §7. Se precisa además que están prohibidas, para los problemas evaluados en §5, `heapq`, `bisect`, `collections` (`deque`, `OrderedDict`, `defaultdict`, `Counter`), `networkx` y cualquier biblioteca de grafos o búsqueda de rutas. `set` queda prohibido en las mismas condiciones que `dict`.

spec-kit y el agente de programación son herramientas de proceso; no pueden introducir dependencias prohibidas en el código entregado. Revisen las dependencias que proponga un plan generado.

6.3 Modo de medición

`--bench` debe incorporar, como mínimo:

- búsqueda de ruta del cazador: un cazador frente a muchos, rango pequeño frente a rango grande, cálculo compartido frente a individual;
- invalidación de rutas: reutilización frente a recálculo siempre, con puertas estables y con puertas inestables;
- consultas al mapa conocido y cálculo de «ir a»;
- resolubilidad: recorrido incremental frente a recorrido repetido por llave;
- línea de tiempo: memoria y tiempo de consulta para al menos tres valores de `K`, y la alternativa sin puntos de control;
- las mediciones de la primera parte repetidas con la carga nueva (§5.6).

Las criptas sintéticas que genere `--bench` deben ser deterministas por semilla.

7. Documentación, entregas y evaluación

7.1 Documento de Decisiones de Diseño

Se actualiza el documento de la primera parte. Cada problema de §5 agrega una sección con la estructura de P1 §8: problema, alternativas consideradas, decisión, análisis de complejidad, evidencia empírica y límites conocidos. Además, cada sección nueva debe:

- enlazar a la especificación y al plan de spec-kit de los que proviene;
- indicar si la decisión final coincide con la del `plan.md` original y, si no, qué medición motivó el cambio;
- en §5.6, mostrar el antes y el después de cualquier decisión de la primera parte que se haya modificado.

7.2 Estructura del repositorio

Como mínimo, el repositorio debe contener:

```
.specify/memory/constitution.md
specs/<NNN-capacidad>/spec.md
specs/<NNN-capacidad>/plan.md
specs/<NNN-capacidad>/tasks.md
DECISIONES.md
PROMPTS_LLM.md
README.md
```

El README se actualiza con: versión de spec-kit, agente utilizado, comandos nuevos (`--verificar`, `--forzar`, `--checkpoint-cada`, `--replay … --hasta`) y cómo ejecutar las pruebas de aceptación.

7.3 Interfaz funcional mínima

Además de lo exigido en P1 §10.2, la interfaz debe permitir:

- ver el mapa conocido;
- ejecutar `ir a <sala>` y ver por qué se detuvo un viaje;
- consultar la línea de tiempo con `historia <t>` e `historia #k`;
- ver el reporte de resolubilidad al iniciar una partida.

La vista debe mostrar los costos de las salidas de la sala actual y distinguir a los cazadores de los demás enemigos.

7.4 Defensa

Además de lo indicado en P1 §9:

- Se escogerá un requisito al azar y el grupo recorrerá su trazabilidad: escenario, tarea, código y prueba (§2.5).
- Se pedirá un cambio pequeño a una regla durante la revisión. El grupo debe mostrar primero la modificación de la especificación y luego la del código.
- Cualquier integrante debe poder explicar cualquier especificación, no solo la de la capacidad que implementó.

7.5 Evaluación

Rubro	Peso dentro del proyecto
Especificaciones, constitución y proceso spec-kit (calidad, trazabilidad, historial)	20 %
Terreno, cazador e invalidación de rutas	20 %
Niebla de guerra e «ir a»	10 %
Verificación de resolubilidad	15 %
Línea de tiempo y puntos de control	15 %
Documento de Decisiones de Diseño	10 %
Rendimiento	10 %

8. Preguntas para arrancar

No deben responderse por escrito. Sirven para orientar la primera discusión técnica del grupo:

- Treinta cazadores persiguen al mismo jugador. ¿Cuántas búsquedas de ruta hacen falta realmente por ciclo?
- Una puerta con cierre automático se cierra lejos de todos los cazadores. ¿Qué rutas calculadas siguen siendo válidas y cómo lo saben sin recalcular?
- ¿Qué diferencia hay entre «el cazador encontró una ruta» y «el cazador encontró la ruta que exige el desempate»?
- ¿Qué información mínima hay que registrar para revertir la exploración de cinco salas con una antorcha encendida?
- Una llave abre una región con otras tres llaves. ¿Qué hace su verificación con las puertas que encontró cerradas antes de tener esa llave?
- Con `K = 1` la consulta histórica es instantánea y con `K` infinito no hay copias. ¿Dónde está el punto razonable y cómo lo van a medir?
- ¿Qué cosas escribirían en `spec.md` que un agente no podría deducir de este enunciado? ¿Qué cosas no deberían escribir allí?
- Si el agente propone usar `heapq` en `plan.md`, ¿en qué fase del flujo debería detectarse y por quién?

Apéndice A. Cambios al contrato de datos

Dato	Origen	Persistencia/validez
`costo` de cada salida	Esqueleto, `GET /criptas/<id>/salas?pagina=n`	Válido mientras coincida la versión de la cripta. Ausente equivale a 100.
`comportamiento: cazador` y `rango`	Catálogo, `GET /catalogo?ids=...`	Válido mientras coincida la versión global del catálogo
Ubicación de llaves	`GET /criptas/<id>/llaves`	Válida mientras coincida la versión de la cripta

Apéndice B. Casos que deben tener un resultado definido

Cada caso debe aparecer como escenario de aceptación en alguna especificación (§2.5).

Terreno y cazador

- Un cazador no tiene ruta abierta hacia el jugador: espera con costo 100.
- La ruta más barata del cazador cuesta más que su `rango`: espera con costo 100.
- Existe una ruta de menos salas pero mayor costo y otra de más salas y menor costo: el cazador sigue la de menor costo.
- Dos vecinas inician rutas de costo mínimo: el cazador va a la de menor id.
- Una puerta de la ruta del cazador se cierra entre dos de sus turnos: en el turno siguiente decide como si calculara desde cero.
- Un actor de velocidad alta atraviesa una salida de costo 1: el intervalo es al menos 1.
- Cambia la velocidad de un actor que atraviesa una salida costosa: el tiempo restante se escala según P1 §2.3.

Niebla de guerra e «ir a»

- `ir a` hacia una sala no explorada: se rechaza y no consume tiempo.
- El mapa recuerda abierta una puerta que en realidad está cerrada: al observarla, el mapa se actualiza y la ruta se recalcula; si no queda ruta, el viaje se detiene sin consumir tiempo adicional.
- `ir a` entra a una sala con enemigos dormidos: los enemigos se activan según P1 §2.4 y el viaje se detiene.
- Con la antorcha encendida, entrar a una sala explora sus vecinas sin activar sus enemigos ni sus trampas.
- Un pergamino revierte una entrada con antorcha encendida: las salas exploradas por la antorcha vuelven a su estado anterior.
- Un pergamino durante un viaje: revierte solo el último paso y el viaje no continúa.

Resolubilidad

- `llave_salida` está detrás de la única puerta que ella misma abre: no resoluble; el reporte señala el ciclo.
- Una llave está en el campo `suelta` de un enemigo errante: la verificación la ubica en la sala inicial del enemigo.
- Existen varios órdenes válidos de obtención de llaves: dos ejecuciones producen el mismo orden.
- Cripta no resoluble sin `--forzar`: la partida no inicia y se muestra el reporte.

Línea de tiempo

- `historia <t>` con `t` mayor que el reloj actual: se rechaza.
- `historia` después de usar un pergamino: no muestra las acciones revertidas.
- Varios eventos ocurren en el mismo instante `t`: `historia <t>` muestra el estado después de todos ellos.
- `historia` no consume tiempo virtual, no se registra en `partida.log` y no modifica el historial de retroceso.
- `--replay partida.log --hasta t` con `t` posterior al final de la partida: muestra el estado final e indica que la partida terminó antes.
- Se carga un archivo de partida de la primera parte: se reporta la incompatibilidad de formato o se migra explícitamente; nunca se carga en silencio.